

universidade CESUMAR – UNICESUMAR
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

KEVIN CALDONAZO DE ALMEIDA

KAIKY DE SOUZA JUNQUEIRA

LUAN PABLOS SANCHES

PSE EM AÇÃO: SISTEMA DE PLANEJAMENTO E ACOMPANHAMENTO DE
AÇÕES DE SAÚDE NA ESCOLA

Atividade de Estudo Programada (AEP) – 1ª Entrega: Documento Escrito

MARINGÁ
2026

KEVIN CALDONAZO DE ALMEIDA – RA: 26015183-2

KAIKY DE SOUZA JUNQUEIRA – RA : 26012834-2

LUAN PABLOS SANCHES – RA: 24150606-2

**PSE EM AÇÃO: SISTEMA DE PLANEJAMENTO E ACOMPANHAMENTO DE
AÇÕES DE SAÚDE NA ESCOLA**

Atividade de Estudo Programada (AEP) apresentada ao curso de Análise e Desenvolvimento de Sistemas da Universidade Cesumar – Unicesumar, série ADSIS2S, como requisito parcial de avaliação do 2º semestre de 2026, referente à 1ª entrega (documento escrito).

MARINGÁ

2026

SUMÁRIO

1 CONTEXTUALIZAÇÃO	4
2 PROBLEMA DE PESQUISA	5
3 OBJETIVOS.....	5
3.1 Objetivo geral	5
3.2 Objetivos específicos.....	5
4 JUSTIFICATIVA.....	6
5 ENGENHARIA DE SOFTWARE.....	7
5.1 Usuários envolvidos	7
5.2 Escopo do sistema	7
5.3 Reflexão sobre uso e limites éticos	8
5.4 Estrutura de dados planejada.....	8
5.5 Requisitos funcionais.....	10
5.6 Requisitos não funcionais.....	11
5.7 Regras de negócio	11
5.8 Processo de desenvolvimento.....	12
5.9 Planejamento das sprints	12
5.10 Riscos do projeto.....	13
6 ALGORITMOS E LÓGICA DE PROGRAMAÇÃO.....	14
6.1 Fluxograma geral do sistema	14
6.2 Fluxograma detalhado: cadastro de ação	15
6.3 Pseudocódigos.....	17
7 REPOSITÓRIO NO GITHUB	24
8 CONSIDERAÇÕES SOBRE ESTA ETAPA	24
REFERÊNCIAS.....	25

1 CONTEXTUALIZAÇÃO

A escola é o lugar onde crianças e jovens passam boa parte do dia e onde muitos hábitos se formam, inclusive os ligados à saúde. Com isso em vista, o governo federal criou o Programa Saúde na Escola (PSE), instituído pelo Decreto nº 6.286, de 5 de dezembro de 2007. O objetivo do programa é contribuir para a formação integral dos estudantes da rede pública de educação básica com ações de promoção, prevenção e atenção à saúde (BRASIL, 2007). Ele atende alunos da educação infantil ao ensino médio, além da educação de jovens e adultos (BRASIL, 2025a).

O programa é conduzido em conjunto pelos Ministérios da Saúde e da Educação, e sua base é a articulação entre a escola e a Atenção Primária à Saúde (BRASIL, 2025a). Em termos simples, a unidade básica de saúde do bairro passa a trabalhar junto com a escola. O Ministério da Saúde ressalta que essa parceria vai além de oferecer serviços no mesmo território: saúde e educação precisam dividir responsabilidades, com apoio dos gestores estaduais e municipais. Na prática, escola, equipe pedagógica e profissionais de saúde precisam planejar e acompanhar as ações juntos.

As ações do PSE tratam de vários temas: alimentação saudável e prevenção da obesidade, atividade física, saúde ambiental, cultura de paz e direitos humanos, prevenção de violências e acidentes, verificação da situação vacinal, saúde sexual e reprodutiva, prevenção ao uso de álcool, tabaco e outras drogas, saúde bucal, auditiva e ocular e saúde mental (BRASIL, 2025b). Segundo o Ministério da Saúde, essas ações devem fazer parte da proposta pedagógica da escola e respeitar a realidade de cada região e a autonomia dos educadores.

Colocar tudo isso em prática, porém, não é simples. Sousa, Esperidião e Medina (2017) avaliaram como funciona a parceria entre saúde e educação no PSE e concluíram que a aproximação entre as duas áreas fortaleceu o programa. Mesmo assim, os setores nem sempre participam de forma equilibrada, e muitas ações continuam pontuais, geralmente em formato de palestra. Lopes, Nogueira e Rocha (2018) chegaram a uma conclusão parecida ao revisar 38 estudos sobre o programa: boa parte das práticas ainda é fragmentada e voltada à prevenção individual.

Um estudo mais recente, de Baroni e Silva (2022), ouviu profissionais da saúde e da educação de uma região periférica da Baixada Santista (SP). As autoras observaram que as ações se concentravam em poucas atividades, como reuniões, encaminhamentos, verificação vacinal e saúde bucal e ocular. Também apontaram que a falta de formação, o desconhecimento sobre a política e a sobrecarga de trabalho atrapalhavam o programa. Para o grupo, esses estudos mostram que as equipes precisam de um jeito simples de registrar o que foi planejado e conferir o que realmente aconteceu.

Um sistema de computador pode ajudar nessa organização, mas exige cuidado. A Lei Geral de Proteção de Dados Pessoais (LGPD) classifica os dados de saúde como dados pessoais sensíveis e traz regras específicas para dados de crianças e adolescentes (BRASIL, 2018). Por isso, o sistema proposto neste trabalho registra apenas informações sobre as ações, como escola, tema e número de participantes, e nunca dados de um aluno específico. Ele também não substitui os profissionais de saúde; sua função é apenas organizar o planejamento e o acompanhamento.

2 PROBLEMA DE PESQUISA

O trabalho parte de um cenário fictício proposto na AEP. A equipe IntegraPSE acompanha várias escolas de um município que participa do PSE e controla as atividades em anotações, formulários e planilhas separadas. Com as informações espalhadas, fica difícil saber quais ações estão programadas, quais já aconteceram, quais foram canceladas e quantos alunos participaram em cada escola. Sem esses números reunidos, a equipe também não consegue avaliar direito o que está dando resultado.

A partir dessa situação, o grupo adotou a seguinte questão norteadora:

Como uma aplicação desenvolvida em linguagem C pode auxiliar uma equipe escolar e de saúde a planejar, registrar e acompanhar ações coletivas do Programa Saúde na Escola, apresentando informações claras e preservando a privacidade dos estudantes?

3 OBJETIVOS

3.1 Objetivo geral

Desenvolver um programa em linguagem C, executado no terminal, que ajude a equipe do PSE a planejar, registrar e acompanhar ações coletivas de saúde nas escolas, mostrando com clareza a situação das ações e a participação do público, sem guardar dados individuais dos estudantes.

3.2 Objetivos específicos

- a) Estudar o Programa Saúde na Escola e as dificuldades de planejamento e acompanhamento apontadas em pesquisas sobre o tema;
- b) Definir quem vai usar o sistema, o que ele fará e o que ficará de fora, e listar os requisitos funcionais e não funcionais;
- c) Representar a lógica do programa em fluxogramas e pseudocódigos;
- d) Implementar em C o cadastro, a listagem, a pesquisa, a atualização de situação e o resumo, com validação dos dados digitados;
- e) Testar e documentar o sistema, preparando o manual do usuário e a apresentação em vídeo.

4 JUSTIFICATIVA

Para o grupo, o trabalho é uma chance de aplicar juntos os conteúdos do semestre. Até aqui, lógica de programação, linguagem C e engenharia de software foram estudadas cada uma na sua disciplina. Neste projeto, é preciso partir de um problema real, escrever os requisitos, desenhar os fluxogramas e escrever o pseudocódigo antes de programar, o que se aproxima do desenvolvimento de software fora da faculdade. O projeto também exige o uso de vetores, struct e funções, e o GitHub para organizar o trabalho em equipe.

Do lado social, o sistema tenta responder a um problema que aparece nos estudos citados: as ações do PSE acontecem de forma solta, e o que foi planejado nem sempre é comparado com o que foi feito (SOUSA; ESPERIDIÃO; MEDINA, 2017; BARONI; SILVA, 2022). Com as ações reunidas em um só lugar e um resumo com o número de ações planejadas, realizadas e canceladas e o percentual de participação, a equipe consegue perceber, por exemplo, que uma escola quase não recebeu ações ou que certo tema acumula cancelamentos.

O grupo também levou em conta os limites do projeto. O sistema não faz diagnóstico nem triagem, não recomenda tratamento e não guarda nome, prontuário ou condição de saúde de nenhum aluno, respeitando a LGPD (BRASIL, 2018). A ideia é que ele sirva como ferramenta de organização para as equipes, sem substituir o trabalho dos profissionais.

5 ENGENHARIA DE SOFTWARE

5.1 Usuários envolvidos

Ao analisar o cenário, o grupo identificou os usuários do Quadro 1. Como o sistema não terá login, todos os usuários diretos usam o programa da mesma forma, no papel de operador. Esse será o único ator nos casos de uso da segunda etapa.

Quadro 1 – Usuários envolvidos e suas necessidades

Usuário	Relação com o sistema	Principal necessidade
Coordenador(a) municipal do PSE (grupo de trabalho intersetorial)	Usuário direto (operador principal)	Ver todas as ações e o resumo de participação para planejar e prestar contas.
Profissionais da Atenção Primária (enfermeiros, dentistas, agentes comunitários de saúde)	Usuário direto (operador)	Cadastrar as ações que conduzem e registrar a participação após a realização.
Gestores escolares e equipe pedagógica	Usuário direto (operador)	Consultar as ações previstas para sua escola e acompanhar sua situação.
Estudantes e comunidade escolar	Beneficiários indiretos (não usam o sistema)	Receber ações de saúde mais bem planejadas, sem ter dados pessoais registrados.

Fonte: elaborado pelos autores (2026).

5.2 Escopo do sistema

O Quadro 2 mostra o que o sistema vai fazer e o que ficou de fora. Os limites levam em conta o nível do segundo semestre e os cuidados éticos pedidos no enunciado.

Quadro 2 – Delimitação do escopo

Dentro do escopo	Fora do escopo
Cadastro de ações coletivas do PSE com código, escola, tema, data prevista, público-alvo, responsável, quantidade prevista e situação.	Cadastro de nome, diagnóstico, prontuário, condição clínica ou qualquer dado individual de estudantes.

Dentro do escopo	Fora do escopo
Listagem, pesquisa por código, escola ou tema e atualização da situação das ações.	Diagnóstico, triagem médica, prescrição ou recomendação de tratamento.
Registro da quantidade efetiva de participantes nas ações realizadas.	Banco de dados, interface gráfica, aplicação web ou bibliotecas externas.
Resumo geral com contagem por situação, total de participantes e percentual de participação.	Controle de usuários e senhas.
Validação das entradas e mensagens claras de confirmação e de erro.	Exclusão de ações e edição completa dos dados (possíveis melhorias futuras).
Armazenamento em memória (vetor de registros) durante a execução.	Gravação em arquivo (melhoria opcional, prevista apenas se houver tempo e se o conteúdo for trabalhado em aula).

Fonte: elaborado pelos autores (2026).

5.3 Reflexão sobre uso e limites éticos

Os principais usuários são profissionais que já têm muito trabalho e nem sempre estão acostumados com computador. Por isso, o menu precisa ser simples e numerado, com mensagens fáceis de entender. O grupo prevê alguns problemas no uso: digitar um número fora do esperado, deixar um campo em branco e perder os dados ao fechar o programa, já que tudo fica só na memória. Nos dois primeiros casos, o programa mostra qual foi o erro e não grava a ação, e o usuário refaz o cadastro pelo menu. Quanto ao terceiro, o manual vai alertar sobre a perda dos dados, e a gravação em arquivo fica como melhoria futura.

Sobre a ética, o sistema guarda apenas informações sobre as ações. O público-alvo é registrado como uma turma (por exemplo, “5º ano – turma A”) e a participação como um número (por exemplo, “40”), sem identificar nenhum aluno. O campo responsável guarda o nome do profissional ou da equipe que conduz a ação. Nos testes e na apresentação, todos os dados serão inventados. **Para a demonstração, o grupo escolheu as temáticas de alimentação saudável e saúde bucal**, mas o sistema aceita ações de qualquer tema do PSE.

5.4 Estrutura de dados planejada

Cada ação precisa guardar vários dados: código, escola, tema, data prevista, público-alvo, responsável e as quantidades de participantes. Em C, o grupo vai reunir esses dados em uma struct e guardar até 100 ações em um vetor. No pseudocódigo, porém, foi usado o Portugol Studio, que não tem o comando registro. Por isso, lá cada informação fica em um vetor próprio (codigos, escolas, temas e assim por diante),

todos com 100 posições, e a mesma posição representa a mesma ação em todos eles. A data prevista é dividida em três campos inteiros (dia, mês e ano), o que permite validar cada número com um simples se. O Quadro 3 mostra os campos e a validação de cada um.

Quadro 3 – Dicionário de dados da ação

Campo	Tipo em C	Descrição	Validação
codigo	int	Identificador da ação	Maior que zero e não repetido
escola	char[40]	Nome da escola	Obrigatório (não vazio)
tema	char[40]	Temática do PSE (ex.: Saúde bucal)	Obrigatório (não vazio)
dia / mes / ano	int (três campos)	Data prevista da ação	Dia de 1 a 31 e mês de 1 a 12
publicoAlvo	char[40]	Grupo atendido (ex.: 5º ano A)	Obrigatório (não vazio)
responsavel	char[40]	Profissional ou equipe responsável	Obrigatório (não vazio)
qtdPrevista	int	Participantes previstos	Maior ou igual a 1
qtdEfetiva	int	Participantes que de fato participaram	Maior ou igual a zero; só informada se realizada
situacao	int	1 = Planejada, 2 = Realizada, 3 = Cancelada	Apenas os valores 1, 2 ou 3

Fonte: elaborado pelos autores (2026).

Em C, a declaração ficará assim:

```
#define MAX_ACOES 100

struct Acao {
    int    codigo;
    char   escola[40];
    char   tema[40];
    int    dia;
    int    mes;
    int    ano;
    char   publicoAlvo[40];
    char   responsavel[40];
    int    qtdPrevista;
    int    qtdEfetiva;
    int    situacao;    /* 1-Planejada 2-Realizada 3-Cancelada */
};

struct Acao acoes[MAX_ACOES];
int totalAcoes = 0;
```

Fonte: elaborado pelos autores (2026).

5.5 Requisitos funcionais

Segundo Sommerville (2011), requisitos funcionais descrevem o que o sistema deve fazer e como deve reagir às entradas do usuário. Os requisitos funcionais do projeto estão no Quadro 4.

Quadro 4 – Requisitos funcionais

ID	Requisito	Descrição	Prioridade
RF01	Cadastrar ação	O sistema deve permitir cadastrar uma ação do PSE informando código, escola, tema, data prevista, público-alvo, responsável, quantidade prevista de participantes e situação inicial.	Essencial
RF02	Listar ações	O sistema deve exibir todas as ações cadastradas de forma organizada, mostrando todos os campos e a situação por extenso.	Essencial
RF03	Pesquisar ações	O sistema deve permitir pesquisar ações pelo código ou pelo nome exato da escola ou do tema, exibindo os registros encontrados.	Essencial
RF04	Atualizar situação	O sistema deve permitir alterar a situação de uma ação, localizada pelo código, para planejada, realizada ou cancelada.	Essencial
RF05	Registrar participação	Quando a situação for definida como realizada, o sistema deve solicitar e registrar a quantidade efetiva de participantes.	Essencial
RF06	Gerar resumo geral	O sistema deve apresentar a quantidade de ações planejadas, realizadas e canceladas, o total de participantes e o percentual de participação em relação ao previsto.	Essencial
RF07	Validar entradas	O sistema deve conferir os dados digitados e recusar o cadastro quando houver código repetido, quantidade inválida, campo de texto em branco, data fora da faixa ou situação inexistente, exibindo uma mensagem que explique o erro.	Essencial
RF08	Controlar o menu	O sistema deve exibir um menu principal repetidamente, permitindo executar várias operações até que o usuário escolha a opção de encerrar.	Essencial

Fonte: elaborado pelos autores (2026).

5.6 Requisitos não funcionais

Já os requisitos não funcionais tratam das restrições e qualidades do sistema como um todo (SOMMERVILLE, 2011). Eles estão no Quadro 5.

Quadro 5 – Requisitos não funcionais

ID	Categoria	Descrição
RNF01	Portabilidade / tecnologia	O sistema deve ser desenvolvido em linguagem C padrão, usando apenas as bibliotecas stdio.h e string.h, e compilar sem erros com o compilador GCC no ambiente definido pelo professor.
RNF02	Interface	O sistema deve funcionar em terminal (modo texto), com menu numerado e textos em língua portuguesa.
RNF03	Privacidade e ética	O sistema não deve armazenar nome, diagnóstico, prontuário, condição clínica ou qualquer dado individual de estudantes, trabalhando somente com dados coletivos e fictícios, em respeito à LGPD.
RNF04	Usabilidade	Toda operação deve exibir mensagem clara de confirmação ou de erro, e as perguntas devem trazer exemplos de preenchimento quando houver formato específico.
RNF05	Armazenamento e capacidade	Os dados devem ser mantidos em memória, em vetor de registros, com capacidade para até 100 ações; ao atingir o limite, o sistema deve avisar o usuário sem encerrar a execução.
RNF06	Manutenibilidade	O código deve ser dividido em funções com responsabilidades claras, uma para cada opção do menu, evitando concentrar a lógica na função main. Devem ser usados somente os recursos estudados na disciplina: variáveis int, float e char, vetores, struct, if/else, switch/case, laços de repetição e funções simples.
RNF07	Robustez	Sempre que o usuário digitar um valor fora do previsto, o sistema deve exibir uma mensagem de erro e voltar ao menu sem gravar dados inválidos.

Fonte: elaborado pelos autores (2026).

5.7 Regras de negócio

Durante o planejamento, surgiram dúvidas que os requisitos não resolviam sozinhos, como o que fazer com os participantes de uma ação cancelada ou como calcular o percentual de participação. As regras do Quadro 6 foram definidas para responder a essas questões.

Quadro 6 – Regras de negócio

ID	Regra
RN01	O código de cada ação é um número inteiro positivo e não pode se repetir.
RN02	A quantidade prevista de participantes deve ser maior que zero.

ID	Regra
RN03	A quantidade efetiva só é informada quando a ação está realizada e deve ser maior ou igual a zero; ela pode superar a quantidade prevista.
RN04	Quando uma ação passa para planejada ou cancelada, a quantidade efetiva é zerada.
RN05	O percentual de participação é calculado apenas sobre as ações realizadas: $(\text{soma das quantidades efetivas} \div \text{soma das quantidades previstas das ações realizadas}) \times 100$. Se não houver ação realizada, o percentual exibido é 0%. Em C, o cálculo usará conversão para float para evitar divisão inteira.
RN06	A pesquisa por escola ou por tema compara o texto digitado com o conteúdo do campo e exibe a ação apenas quando os dois são iguais. A busca por parte do texto fica como melhoria futura.
RN07	A data prevista é formada por três números inteiros. O sistema aceita dia de 1 a 31 e mês de 1 a 12, mas não verifica quantos dias tem cada mês.

Fonte: elaborado pelos autores (2026).

5.8 Processo de desenvolvimento

O grupo vai usar o modelo incremental. Nele, o sistema é construído em partes: cada versão acrescenta funcionalidades à anterior, o que permite testar desde cedo e corrigir problemas antes que cresçam (SOMMERVILLE, 2011; PRESSMAN; MAXIM, 2016). O trabalho será dividido em sprints de uma semana, e cada sprint deve terminar com algo pronto, seja uma parte do documento, seja uma parte do programa funcionando. Tudo será enviado ao GitHub por meio de commits.

No começo de cada semana, o grupo vai se reunir rapidamente para dividir as tarefas e, no fim, conferir o que foi concluído e ajustar a sprint seguinte. A implementação começa pela struct e pelo menu. Depois vêm o cadastro e a listagem, que são necessários para testar o restante, e por último a pesquisa, a atualização e o resumo.

5.9 Planejamento das sprints

O Quadro 7 mostra o planejamento semana a semana. As datas ainda serão ajustadas de acordo com os prazos definidos pelo professor.

Quadro 7 – Planejamento das sprints semanais

Sprint	Foco	Atividades	Entregável
Semana 1	Levantamento	Estudo do enunciado e das fontes sobre o PSE; definição do problema, dos objetivos e da justificativa; criação do repositório no GitHub.	Contextualização, problema, objetivos e repositório criado

Sprint	Foco	Atividades	Entregável
Semana 2	Requisitos e modelagem	Identificação de usuários; delimitação do escopo; requisitos funcionais e não funcionais; regras de negócio; riscos; estrutura de dados; fluxogramas.	Requisitos, escopo, riscos e fluxogramas
Semana 3	Algoritmos e entrega 1	Pseudocódigos; revisão geral do documento e das referências; README e pasta docs no GitHub.	1ª entrega (documento e GitHub)
Semana 4	Implementação – incremento 1	Struct, vetor, funções auxiliares de leitura, menu principal, cadastro com validações e listagem.	Programa com menu, cadastro e listagem
Semana 5	Implementação – incremento 2	Pesquisa por código, escola e tema; atualização de situação com registro de participantes; resumo geral.	Programa com as seis funcionalidades
Semana 6	Testes	Elaboração e execução de casos de teste (entradas válidas, inválidas e limites); correção de erros; teste de compilação no ambiente do professor.	Registro dos testes e código corrigido
Semana 7	Documentação	Diagrama e descrição dos casos de uso; atualização dos requisitos; manual do usuário com capturas de tela; considerações finais.	Documento final revisado
Semana 8	Apresentação e entrega 2	Roteiro e gravação do vídeo de até 7 minutos com todos os integrantes; revisão final do repositório.	2ª entrega (código, documento e vídeo)

Fonte: elaborado pelos autores (2026).

5.10 Riscos do projeto

O Quadro 8 lista os riscos que o grupo identificou e o que pretendemos fazer para evitá-los ou diminuir seus efeitos.

Quadro 8 – Riscos do projeto

Risco	Probab.	Impacto	Ação de mitigação
Atraso ou indisponibilidade de um integrante	Média	Alto	Dividir tarefas por sprint, registrar o andamento no GitHub e manter mais de um integrante conhecendo cada parte.
Dificuldades técnicas em C (strings, leitura com scanf, lixo no buffer de entrada)	Alta	Médio	Criar cedo as funções auxiliares de leitura, testar cada função isoladamente e consultar o material da disciplina.
Programa não compilar no ambiente do professor	Média	Alto	Usar apenas bibliotecas padrão, evitar recursos específicos de sistema operacional (como conio.h) e testar com GCC antes da entrega.
Perda de código ou versões conflitantes	Baixa	Alto	Versionar todo o trabalho no GitHub, com commits frequentes e descritivos.

Risco	Probab.	Impacto	Ação de mitigação
Interpretação equivocada do enunciado	Média	Médio	Comparar os artefatos com o escopo mínimo e os critérios de avaliação ao final de cada sprint e tirar dúvidas com o professor.
Inclusão indevida de dados sensíveis nos testes	Baixa	Alto	Usar apenas dados fictícios e coletivos e revisar os campos e os exemplos antes de cada entrega.
Acúmulo de atividades de outras disciplinas	Alta	Médio	Reservar folga na semana de testes e antecipar tarefas de documentação sempre que possível.

Fonte: elaborado pelos autores (2026).

6 ALGORITMOS E LÓGICA DE PROGRAMAÇÃO

Antes de programar, o grupo representou a lógica do sistema em fluxogramas e pseudocódigos, o que ajuda a enxergar a sequência de passos e as decisões do algoritmo (FORBELLONE; EBERSPÄCHER, 2005). Os símbolos usados nos fluxogramas estão no Quadro 9.

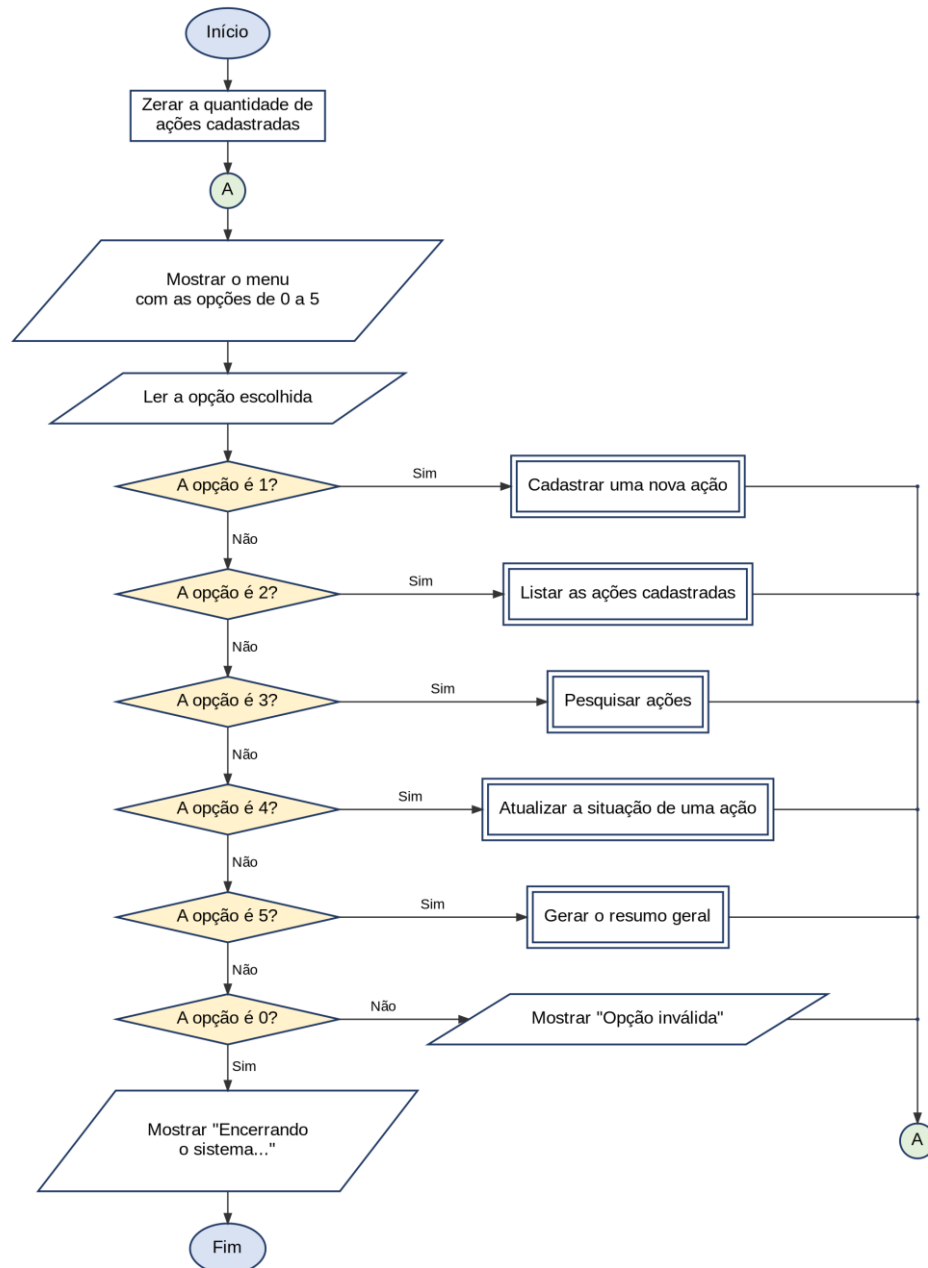
Quadro 9 – Simbologia utilizada nos fluxogramas

Símbolo	Significado
Elipse	Terminal: início ou fim do algoritmo ou da sub-rotina.
Retângulo	Processamento: atribuição ou cálculo.
Retângulo com borda dupla	Processo predefinido: chamada de uma função (sub-rotina) descrita separadamente.
Paralelogramo	Entrada ou saída de dados (leitura do teclado ou exibição na tela).
Losango	Decisão: condição com saídas “Sim” e “Não”.
Círculo com letra	Conector: liga partes do fluxograma (A retorna ao menu; B continua o fluxo na parte seguinte).
Seta	Indica o sentido em que o algoritmo é executado.

Fonte: elaborado pelos autores (2026).

6.1 Fluxograma geral do sistema

A Figura 1 mostra o funcionamento geral do programa. Primeiro, o contador de ações é zerado. Em seguida, o menu é exibido e o programa lê a opção escolhida. Cada opção válida chama a função correspondente e, quando ela termina, o fluxo volta ao menu pelo conector A. Uma opção que não existe gera mensagem de erro, e a opção 0 encerra o programa. No código, isso corresponde a um laço repita...até que se repete enquanto a opção for diferente de zero.

Figura 1 – Fluxograma geral do sistema

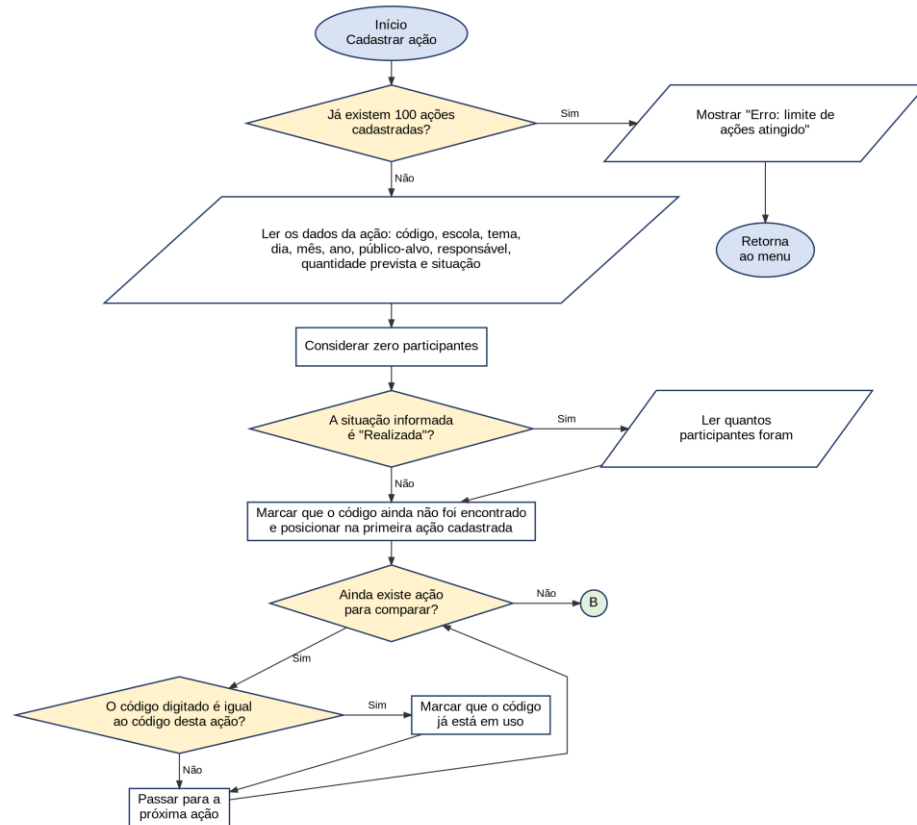
Fonte: elaborado pelos autores (2026).

6.2 Fluxograma detalhado: cadastro de ação

Para o fluxograma detalhado, o grupo escolheu o cadastro, que é a operação com mais validações. O programa verifica primeiro se ainda há espaço na lista. Depois lê todos os dados da ação e, se ela já estiver realizada, pergunta também quantas pessoas participaram. Em seguida percorre as ações já cadastradas, com um laço, para descobrir se o código digitado já está em uso. Por fim, confere um a um os dados informados: se algum estiver errado, mostra a mensagem correspondente e marca os dados como inválidos. A ação só é guardada quando nenhum erro é encontrado.

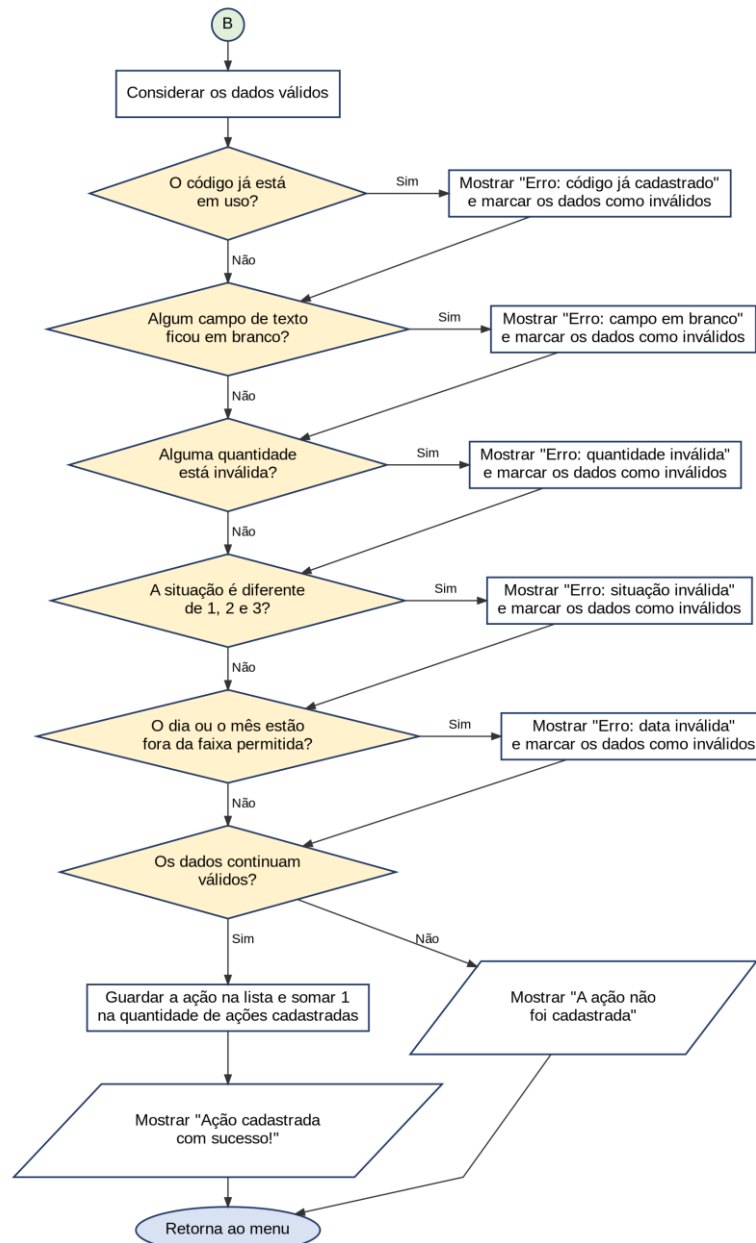
Como o fluxograma ficou grande, ele foi dividido em duas partes ligadas pelo conector B (Figuras 2 e 3).

Figura 2 – Fluxograma do cadastro de ação (parte 1)



Fonte: elaborado pelos autores (2026).

Figura 3 – Fluxograma do cadastro de ação (parte 2)



Fonte: elaborado pelos autores (2026).

Os pseudocódigos do Quadro 11 ao Quadro 14 detalham cada uma dessas operações.

6.3 Pseudocódigos

Os pseudocódigos foram escritos no Portugol Studio e podem ser executados nele. O grupo usou apenas o que já foi visto em aula: variáveis inteiro, real e cadeia, vetores, decisões com se/senão e escolha/caso, os laços para e faça...enquanto e funções simples. Como o Portugol Studio não possui o comando registro, cada informação da ação fica em um vetor separado, e a posição *i* representa a mesma

ação em todos eles. O \n no fim das mensagens serve para pular a linha. Em C, esses vetores serão substituídos por um único vetor de struct, e a comparação entre dois textos, feita aqui com o operador ==, passará a usar a função strcmp, da biblioteca string.h. O Quadro 10 mostra os vetores e o menu principal; as funções dos Quadros 11 a 14 devem ser copiadas para dentro do bloco programa { }, antes da última chave, para que o algoritmo seja executado.

Quadro 10 – Pseudocódigo dos vetores e do menu principal

```
programa {

    const inteiro MAX = 100

    // um vetor para cada informacao da acao
    inteiro codigos[MAX]
    cadeia  escolas[MAX]
    cadeia  temas[MAX]
    inteiro dias[MAX]
    inteiro meses[MAX]
    inteiro anos[MAX]
    cadeia  publicos[MAX]
    cadeia  responsaveis[MAX]
    inteiro previstos[MAX]
    inteiro efetivos[MAX]
    inteiro situacoes[MAX]
    inteiro totalAcoes = 0

    funcao inicio() {
        inteiro opcao

        faca {
            escreva("==== PSE EM ACAO =====\n")
            escreva("1 - Cadastrar acao\n")
            escreva("2 - Listar acoes\n")
            escreva("3 - Pesquisar acoes\n")
            escreva("4 - Atualizar situacao\n")
            escreva("5 - Gerar resumo\n")
            escreva("0 - Sair\n")
            escreva("Escolha uma opcao: ")
            leia(opcao)

            escolha (opcao) {
                caso 1:
                    cadastrarAcao()
                    pare
                caso 2:
                    listarAcoes()
                    pare
                caso 3:
                    pesquisarAcoes()
                    pare
                caso 4:
                    atualizarSituacao()
                    pare
                caso 5:
                    gerarResumo()
                    pare
                caso 0:
                    escreva("Encerrando o sistema...\n")
                    pare
                caso contrario:
                    escreva("Opcao invalida! Digite um numero de 0 a 5.\n")
            }
        }
    }
}
```

```

    }
    } enquanto (opcao != 0)
  }
}

```

Fonte: elaborado pelos autores (2026).

Quadro 11 – Pseudocódigo da função que exibe uma ação

```

funcao mostrarAcao(inteiro i) {
  escreva("Codigo: ", codigos[i], "\n")
  escreva("Escola: ", escolas[i], "\n")
  escreva("Tema: ", temas[i], "\n")
  escreva("Data prevista: ", dias[i], "/", meses[i], "/", anos[i], "\n")
  escreva("Publico-alvo: ", publicos[i], "\n")
  escreva("Responsavel: ", responsaveis[i], "\n")
  escreva("Participantes previstos: ", previstos[i], "\n")

  escolha (situacoes[i]) {
    caso 1:
      escreva("Situacao: Planejada\n")
      pare
    caso 2:
      escreva("Situacao: Realizada\n")
      escreva("Participantes que foram: ", efetivos[i], "\n")
      pare
    caso 3:
      escreva("Situacao: Cancelada\n")
      pare
  }

  escreva("-----\n")
}

```

Fonte: elaborado pelos autores (2026).

Quadro 12 – Pseudocódigo do cadastro de ação

```

funcao cadastrarAcao() {
  inteiro i, existe, valido
  inteiro codigo, dia, mes, ano, previsto, efetivo, situacao
  cadeia escola, tema, publico, responsavel

  se (totalAcoes >= MAX) {
    escreva("Erro: o sistema ja tem 100 acoes cadastradas.\n")
  } senao {
    escreva("Digite o codigo da acao: ")
    leia(codigo)
    escreva("Digite o nome da escola: ")
    leia(escola)
    escreva("Digite o tema: ")
    leia(tema)
    escreva("Digite o dia: ")
    leia(dia)
    escreva("Digite o mes: ")
    leia(mes)
    escreva("Digite o ano: ")
    leia(ano)
    escreva("Digite o publico-alvo: ")
    leia(publico)
    escreva("Digite o responsavel: ")
    leia(responsavel)
    escreva("Digite a quantidade prevista: ")
    leia(previsto)
    escreva("Situacao (1-Planejada, 2-Realizada, 3-Cancelada): ")
    leia(situacao)
  }
}

```

```

    efetivo = 0
    se (situacao == 2) {
        escreva("Quantos participantes foram? ")
        leia(efetivo)
    }

    // procura se o codigo ja foi usado
    existe = 0
    para (i = 0; i < totalAcoes; i++) {
        se (codigos[i] == codigo) {
            existe = 1
        }
    }

    // confere os dados digitados
    valido = 1

    se (existe == 1) {
        escreva("Erro: ja existe uma acao com esse codigo.\n")
        valido = 0
    }
    se (escola == "" ou tema == "" ou publico == "" ou responsavel == "") {
        escreva("Erro: os campos de texto nao podem ficar em branco.\n")
        valido = 0
    }
    se (previsto <= 0 ou efetivo < 0) {
        escreva("Erro: quantidade invalida.\n")
        valido = 0
    }
    se (situacao < 1 ou situacao > 3) {
        escreva("Erro: situacao invalida.\n")
        valido = 0
    }
    se (dia < 1 ou dia > 31 ou mes < 1 ou mes > 12) {
        escreva("Erro: data invalida.\n")
        valido = 0
    }

    se (valido == 1) {
        codigos[totalAcoes] = codigo
        escolas[totalAcoes] = escola
        temas[totalAcoes] = tema
        dias[totalAcoes] = dia
        meses[totalAcoes] = mes
        anos[totalAcoes] = ano
        publicos[totalAcoes] = publico
        responsaveis[totalAcoes] = responsavel
        previstos[totalAcoes] = previsto
        efetivos[totalAcoes] = efetivo
        situacoes[totalAcoes] = situacao

        totalAcoes = totalAcoes + 1
        escreva("Acao cadastrada com sucesso!\n")
    } senao {
        escreva("A acao nao foi cadastrada.\n")
    }
}

```

Fonte: elaborado pelos autores (2026).

Quadro 13 – Pseudocódigo da listagem e da pesquisa

```

funcao listarAcoes() {
    inteiro i

    se (totalAcoes == 0) {

```

```

        escreva("Nenhuma acao cadastrada.\n")
    } senao {
        escreva("Total de acoes: ", totalAcoes, "\n")
        para (i = 0; i < totalAcoes; i++) {
            mostrarAcao(i)
        }
    }
}

funcao pesquisarAcoes() {
    inteiro i, tipo, codigo, encontrados
    cadeia termo

    escreva("Pesquisar por: 1 -Codigo, 2 - Escola, 3 - Tema\n")
    escreva("Escolha uma opcao: ")
    leia(tipo)
    encontrados = 0

    se (tipo == 1) {
        escreva("Digite o codigo: ")
        leia(codigo)
        para (i = 0; i < totalAcoes; i++) {
            se (codigos[i] == codigo) {
                mostrarAcao(i)
                encontrados = encontrados + 1
            }
        }
    }

    se (tipo == 2 ou tipo == 3) {
        escreva("Digite o texto que quer procurar: ")
        leia(termo)
        para (i = 0; i < totalAcoes; i++) {
            se (tipo == 2 e escolas[i] == termo) {
                mostrarAcao(i)
                encontrados = encontrados + 1
            }
            se (tipo == 3 e temas[i] == termo) {
                mostrarAcao(i)
                encontrados = encontrados + 1
            }
        }
    }

    se (tipo < 1 ou tipo > 3) {
        escreva("Opcao invalida.\n")
    } senao {
        se (encontrados == 0) {
            escreva("Nenhuma acao encontrada.\n")
        }
    }
}

```

Fonte: elaborado pelos autores (2026).

Quadro 14 – Pseudocódigo da atualização de situação e do resumo geral

```

funcao atualizarSituacao() {
    inteiro i, codigo, pos, novaSituacao, efetivo

    se (totalAcoes == 0) {
        escreva("Nenhuma acao cadastrada.\n")
    } senao {
        escreva("Digite o codigo da acao: ")
        leia(codigo)

        // procura a posicao da acao nos vetores
    }
}

```

```

    pos = -1
    para (i = 0; i < totalAcoes; i++) {
        se (codigos[i] == codigo) {
            pos = i
        }
    }

    se (pos == -1) {
        escreva("Erro: acao nao encontrada.\n")
    } senao {
        mostrarAcao(pos)
        escreva("Nova situacao (1-Planejada, 2-Realizada, 3-Cancelada): ")
        leia(novaSituacao)

        se (novaSituacao < 1 ou novaSituacao > 3) {
            escreva("Erro: situacao invalida.\n")
        } senao {
            situacoes[pos] = novaSituacao
            efetivos[pos] = 0

            se (novaSituacao == 2) {
                escreva("Quantos participantes foram? ")
                leia(efetivo)
                se (efetivo < 0) {
                    escreva("Erro: valor negativo. Foi gravado zero.\n")
                    efetivo = 0
                }
                efetivos[pos] = efetivo
            }

            escreva("Situacao atualizada!\n")
        }
    }
}

funcao gerarResumo() {
    inteiro i, planejadas, realizadas, canceladas
    inteiro totalParticipantes, totalPrevisto
    real    percentual

    planejadas = 0
    realizadas = 0
    canceladas = 0
    totalParticipantes = 0
    totalPrevisto = 0

    para (i = 0; i < totalAcoes; i++) {
        escolha (situacoes[i]) {
            caso 1:
                planejadas = planejadas + 1
                pare
            caso 2:
                realizadas = realizadas + 1
                totalParticipantes = totalParticipantes + efetivos[i]
                totalPrevisto = totalPrevisto + previstos[i]
                pare
            caso 3:
                canceladas = canceladas + 1
                pare
        }
    }

    percentual = 0.0
    se (totalPrevisto > 0) {
        percentual = (totalParticipantes * 100.0) / totalPrevisto
    }
}

```

```

escreva("==== RESUMO GERAL ====\\n")
escreva("Total de acoes: ", totalAcoes, "\\n")
escreva("Planejadas: ", planejadas, "\\n")
escreva("Realizadas: ", realizadas, "\\n")
escreva("Canceladas: ", canceladas, "\\n")
escreva("Total de participantes: ", totalParticipantes, "\\n")
escreva("Percentual de participacao: ", percentual, "%\\n")
}

```

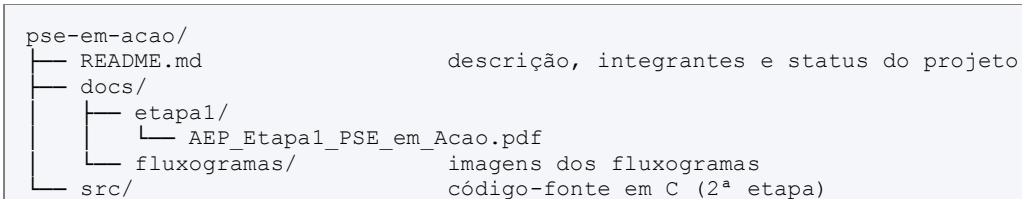
Fonte: elaborado pelos autores (2026).

7 REPOSITÓRIO NO GITHUB

O projeto fica em um repositório público no GitHub, com o README (descrição do projeto e integrantes), a pasta docs, com os documentos de cada etapa, e a pasta src, que vai receber o código na segunda etapa. A organização está no Quadro 15.

Link do repositório: <https://github.com/KaikySJ/pse-em-acao.git>

Quadro 15 – Estrutura do repositório



Fonte: elaborado pelos autores (2026).

8 CONSIDERAÇÕES SOBRE ESTA ETAPA

Nesta primeira etapa, o grupo estudou o PSE e entendeu melhor por que é difícil planejar e acompanhar as ações na prática. Com base nisso, definimos os usuários, o escopo, os requisitos, as regras, os riscos e as sprints, e desenhamos a lógica das principais operações em fluxogramas e pseudocódigos. Ao montar os algoritmos, o grupo optou por usar somente os recursos já vistos em aula, o que mantém o sistema dentro do nível do segundo semestre e torna a implementação mais segura.

Na segunda etapa, esse planejamento vai virar o programa em C, junto com os testes, os casos de uso e o manual do usuário. Se algum requisito mudar durante a implementação, a mudança será registrada e explicada no documento final.

REFERÊNCIAS

BARONI, Juliane Gonçallo; SILVA, Carla Cilene Baptista da. Percepção de profissionais da saúde e da educação sobre o Programa Saúde na Escola. **Saúde em Debate**, Rio de Janeiro, v. 46, n. especial 3, p. 103-115, 2022. DOI: 10.1590/0103-11042022e307.

BRASIL. **Decreto nº 6.286, de 5 de dezembro de 2007**. Institui o Programa Saúde na Escola – PSE, e dá outras providências. Diário Oficial da União: Brasília, DF, 6 dez. 2007.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Diário Oficial da União: Brasília, DF, 15 ago. 2018.

BRASIL. Ministério da Saúde. **O que é o Programa Saúde na Escola (PSE)?** Brasília, DF: Ministério da Saúde, 2025a. Disponível em: <https://www.gov.br/saude/pt-br/composicao/saps/pse/faq/o-que-e-o-programa>. Acesso em: 10 set. 2026.

BRASIL. Ministério da Saúde. **Quais são as ações do PSE?** Brasília, DF: Ministério da Saúde, 2025b. Disponível em: <https://www.gov.br/saude/pt-br/composicao/saps/pse/faq/quais-sao-as-acoes-do>. Acesso em: 10 set. 2026.

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. **Lógica de programação**: a construção de algoritmos e estruturas de dados. 3. ed. São Paulo: Pearson Prentice Hall, 2005.

LOPES, Iraneide Etelvina; NOGUEIRA, Júlia Aparecida Devidé; ROCHA, Dais Gonçalves. Eixos de ação do Programa Saúde na Escola e Promoção da Saúde: revisão integrativa. **Saúde em Debate**, Rio de Janeiro, v. 42, n. 118, p. 773-789, 2018. DOI: 10.1590/0103-1104201811819.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011.

SOUSA, Marta Caires de; ESPERIDIÃO, Monique Azevedo; MEDINA, Maria Guadalupe. A intersetorialidade no Programa Saúde na Escola: avaliação do processo político-gerencial e das práticas de trabalho. **Ciência & Saúde Coletiva**, Rio de Janeiro, v. 22, n. 6, p. 1781-1790, 2017. DOI: 10.1590/1413-81232017226.24262016.